

Lab session 1: Getting started

EMBEDDED SIGNAL PROCESSING SYSTEMS HT 2018

Instructions

- This lab session is intended for you to get started with
 - (1). *AVR UC3 Xplained* evaluation kit (hardware)
 - (2). *Atmel Studio 7* (software) – the integrated development platform (IDP) for developing and debugging Atmel AVR® microcontroller applications, available on <http://www.microchip.com/avr-support/atmel-studio-7>
 - (3). *BatchISP* (software) – for loading program/firmware onto a MCU board
 - (4). *PuTTY* (an SSH & telnet client) – for the communications between microcontrollers (MCU) and computers (e.g., laptops, desktops) via universal synchronous/asynchronous receiver/transmitter (USART)

that are needed for the course. The details on (1) – (3) are available in [M8, M9, M10], and on the web site: <http://www.atmel.com/products/microcontrollers/avr/default.aspx>. Many good videos on Getting Started with Atmel AVR and Atmel Studio are available on www.youtube.com by searching "Getting Started with AVR" and "Getting Started with Atmel Studio 7"

- Two example projects (programs) are to be implemented and tested
 - (i). Test LEDs on the evaluation kit
 - (ii). Test USART on the evaluation kit
- Optional exercises for you to practice programming embedded systems in C
- The lab session is done in groups of two students.

Devices and Equipment:

- *AVR UC3 Xplained* evaluation kit
- a USB-RS232 adapter cable
- a computer installed with MATLAB, Atmel Studio 7, Flip (Flip 3.4.7 is the latest version) and Putty; Use of your own laptop is highly recommended if you would like to work on the embedded systems at home as well

Contents

Lab session 1: Getting started	1
1. Atmel AVR UC3 microcontrollers [M10, M15]	2
2. AVR UC3-A3 Xplained evaluation board [M8, M9]	2
2.1 Power supply	3
2.2 Preprogrammed firmware	3
2.3 Programming the UC3-A3 Xplained evaluation kit	3
2.4 Real-time signal processing	3
3. Atmel Studio 7	4
3.1 Getting started	4
4. batchisp	9
5. PuTTY and USART	10
6. Exercises	16
Appendix A. Block diagram and key parameters of AT32UC3A3256 microcontroller	18
References	20

1. Atmel AVR UC3 microcontrollers [M10, M15]

AVR UC3 microcontrollers are 32-bit microcontrollers having a 16-512KB Flash, 48-144 pins, and up to 66 MHz and 1.5 MIPS/MHz. They have different series of A0/A1, A3/A4, B, C, D, L and audio, which are designed for different applications. Each series has a few different models. The A3/A4 series has models of AT32UC3A364, AT32UC3A3128, and AT32UC3A3256, which have 64, 128, and 256 KB flashes, respectively.

2. AVR UC3-A3 Xplained evaluation board [M8, M9]

Refer to Getting Started Guide and Hardware User's Guide [M8, M9].

The Atmel® UC3-A3 Xplained evaluation board [M8, M9] (Fig. 1) is intended for evaluating the Atmel AT32UC3A3256 microcontroller that has a 256 KB Flash, and a 66 MHz speed. It is used in this course to implement real-time digital signal processing.

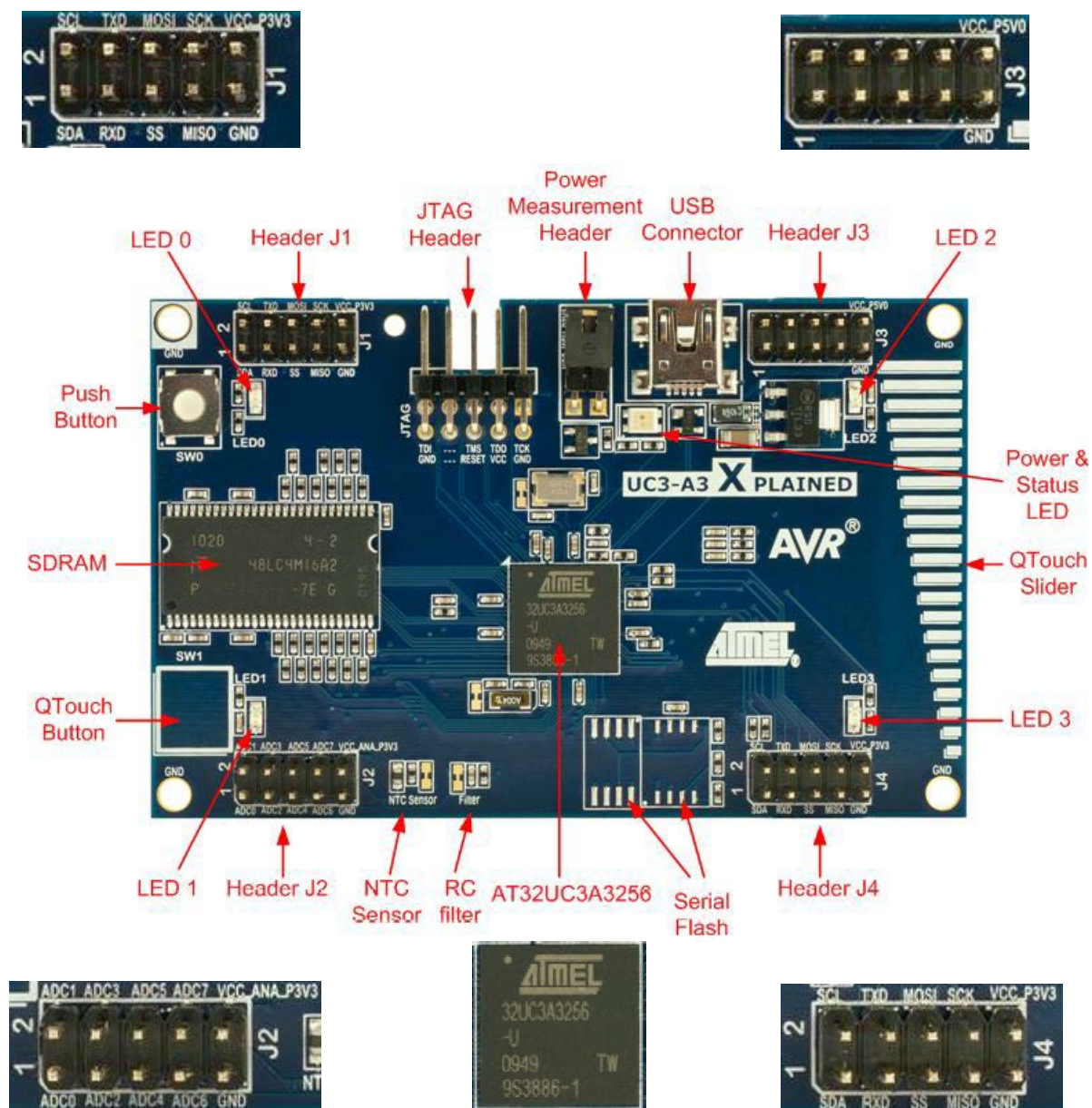


Fig. 1. Overview of the UC3-A3 Xplained evaluation board

Fig. 1 shows the available features of the Atmel® UC3-A3 Xplained evaluation board:

- Atmel® AT32UC3A3256 microcontroller
- High-speed USB interface for transfer data
- 64MBit SDRAM to which data can be read/written
- Analog input (to ADC)
- Temperature sensor which is read with the ADC
- RC filter
- 1 push button (SW0) to interact with the microcontroller
- 4 LEDs (LED 0 – LED 4) which can be used to show status information
- 4 expansion headers (J1 –J4)
- Footprints for external memory
- Atmel AT45DB series DataFlash® serial flash
- Atmel AT25DF series industry standard serial flash
- 1 Atmel QTouch® slider and 1 QTouch button (SW1) that are detected with the Atmel QTouch library

2.1 Power supply

The kit needs an external power supply that can deliver 5V and up to 500mA. The actual current requirement for the board is much less than 500mA but in order to be able to power possible expansion boards this margin is needed.

The power can be applied to the board either via the USB connector or on pin 10 on the header J3. The USB connector is the preferred input because then it is possible to connect expansion boards on top of the J3 header. When the evaluation board is connected to a PC with a USB cable (Standard-A to Mini-B), the power is applied and the Power/Status LED will light up green.

The 5V is regulated down to 3.3V with an on board LDO regulator, which provides power to the entire board. Expansion top boards that require 5V will get this from the header J3 pin 10.

2.2 Preprogrammed firmware

The Atmel AT32UC3A3256 on a brand new Atmel UC3-A3 Xplained evaluation kit is preprogrammed with a USB bootloader and a default firmware.

The preprogrammed firmware in the AT32UC3A3256 is a simple demonstration of the QTouch button and QTouch slider. It runs once the evaluation board is connected to a PC with a USB cable (Standard-A to Mini-B). Touching the SW1 QTouch button will cause LED1 to light up. Moving a finger along the QTouch slider will cause either LED2 or LED3 to light up, depending on the position of the finger.

2.3 Programming the UC3-A3 Xplained evaluation kit

To program the evaluation kit, a USB bootloader (or an external programming tool, i.e., a programmer or a debugger like AVR JTAGICE mkII, AVR Dragon™, or AVR ONE!) is needed

To evoke the boot loader, push the physical push button (SW0) during powerup; that is, push the button, and then connect a USB cable to the kit. Programming can be performed by using the BatchISP command line tool, which is part of the FLIP installation. BatchISP can also run under Atmel AVR32 Studio, which has a GUI interface.

Once the boot loader is active, it will appear as a USB DFU class device, and Windows will ask for a driver. The driver can be found under the FLIP installation directory. When the driver is installed and the boot loader is working, one can see “Atmel USB Devices” in the list of “Device Manager”.

2.4 Real-time signal processing

The Atmel UC3-A3 Xplained evaluation kit has sufficient resource to realize real-time signal processing. It has a 32-bit AT32UC3A3256 microcontroller which is used as DSP, 8 channels for analog-to-digital conversion (ADC), and two channels available for digital- analog conversion (DAC)

[M10]. The most important information on these is given in Appendix A, and more details are available in [M10].

3. Atmel Studio 7

Atmel® Studio 7 is the integrated development platform (IDP) for developing and debugging Atmel ARM® Cortex™-M processor-based and Atmel AVR® microcontroller applications. The Atmel Studio 7 IDP gives you a seamless and easy-to-use environment to write, build and debug your applications written in C/C++ or assembly code. Atmel Studio 7 supports all 8- and 32-bit AVR, the new SoC wireless family, SAM3 and SAM4 microcontrollers, and connects seamlessly to Atmel debuggers and development kits.

Atmel Gallery is an online apps store built in to Studio 7, allowing you to purchase both in-house and third-party development tools and embedded software. Atmel Spaces is a collaborative workspace where you can securely share embedded design and track progress of projects with your peers.

3.1 Getting started

Step 1: click on the Atmel Studio 7.0 shortcut (Fig. 2).

The studio is starting up with a picture (Fig. 3) showing on the screen. Once started, the start page is displayed (Fig. 4). From within this page you can create new projects, reopen recently used projects, and get access to various links for learning and exploring Atmel Studio. The Start page can also be accessed from **View→Start Page**, or **Alt+V+G**.

Step 2: click on **New Project** in the project related section in Start Page (Fig. 5).

The New Project window appears (Fig. 6).

Step 3: click on **C/C++** in the left pane of the **New Project** window (Fig. 6) and then select GCC C ASF Board Project C/C++ from within the middle pane (Fig. 6).

Give a solution name (e.g., Test_LED) in Solution name and a project name (e.g., Test_LED) in Name (the two names are allowed to be different) and then press OK (Fig. 6). The **Board Selection** window appears (Fig. 7(a)). In this window, first select **Select By Board** on the top in the upper left pane, then select Board on the top of the right pane, and finally choose UC3-Xplained – AT32UC3A3256 in the upper left pane. After this, the **Board Selection** window has a look as shown in Fig. 7(b). After clicking on Ok, the project Test_LED is created. The Test_LED window appears, looking like Fig. 8(a) after choosing Test_LED in the item Project: In the upper right pane, click on src, and then choose main.c. The Test_LED window has appearance shown in Fig. 8(b). It shows that the automatically created Test_LED project has a basic program structure and a minimum number of files needed to start a project (a C/C++ program) that works with the ASF application builder.



Atmel Studio 7.0

Fig. 2. Atmel Studio 7.0 shortcut



Fig. 3. Atmel Studio 7.0 is starting.

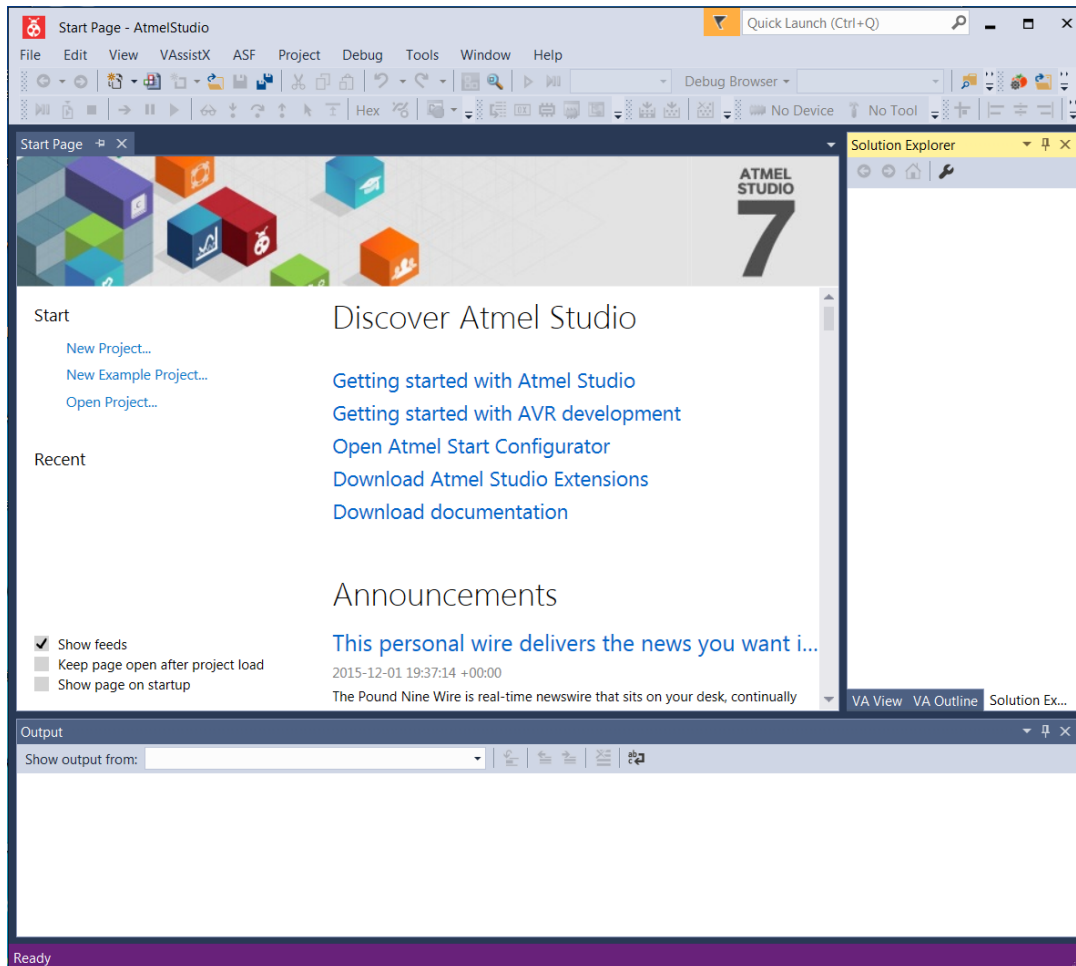


Fig. 4. The Start Page of Atmel Studio 7

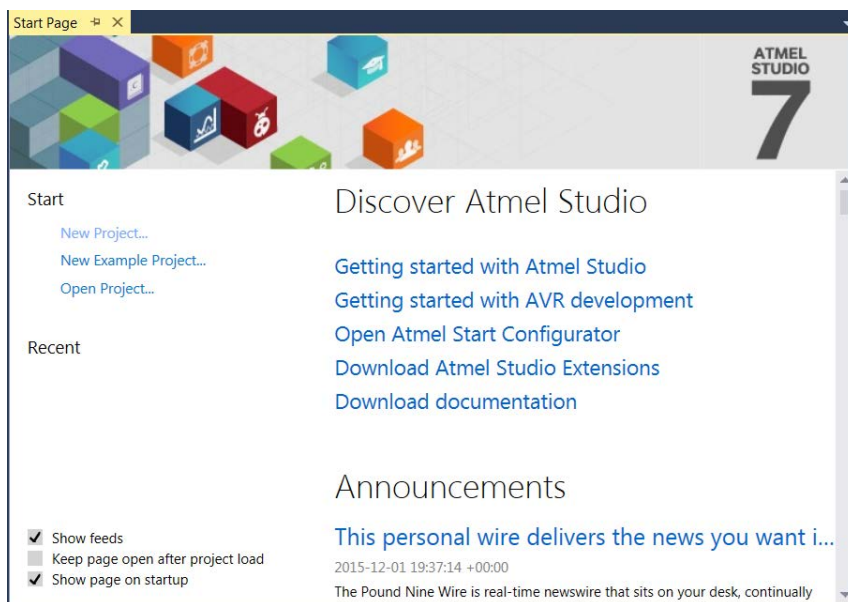


Fig. 5. The Project related section of the Start Page

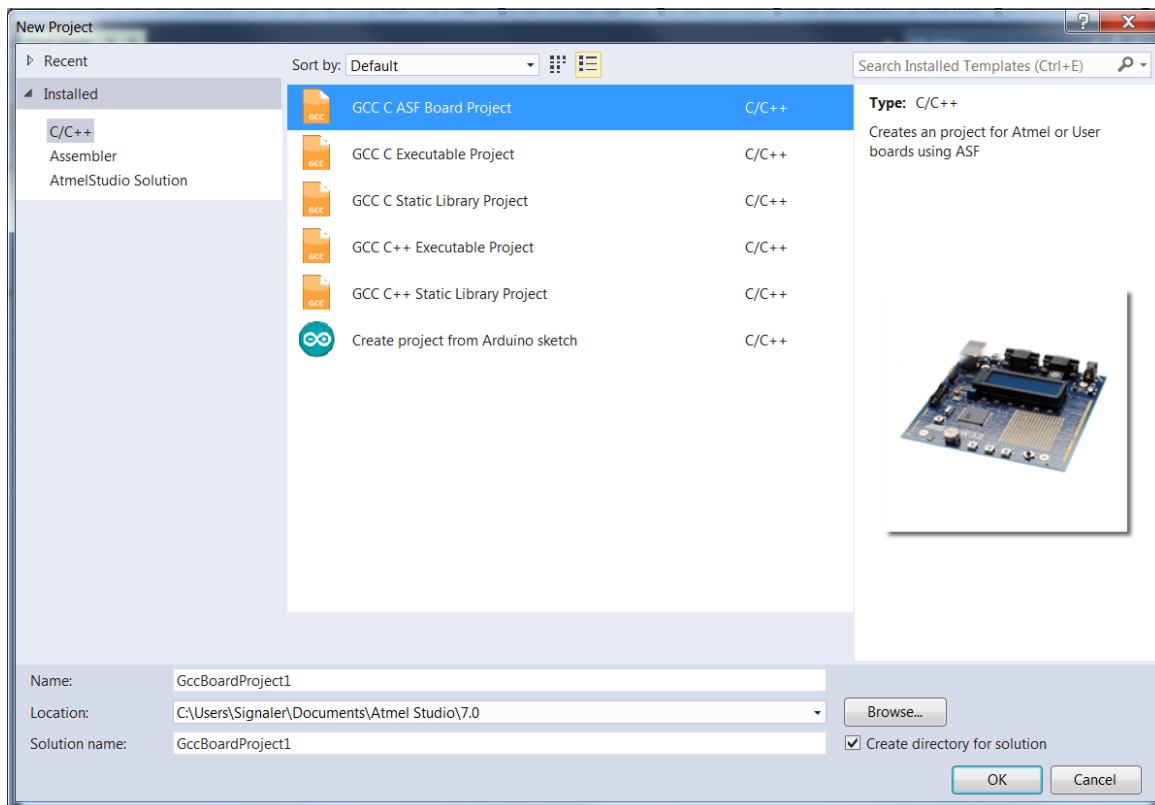
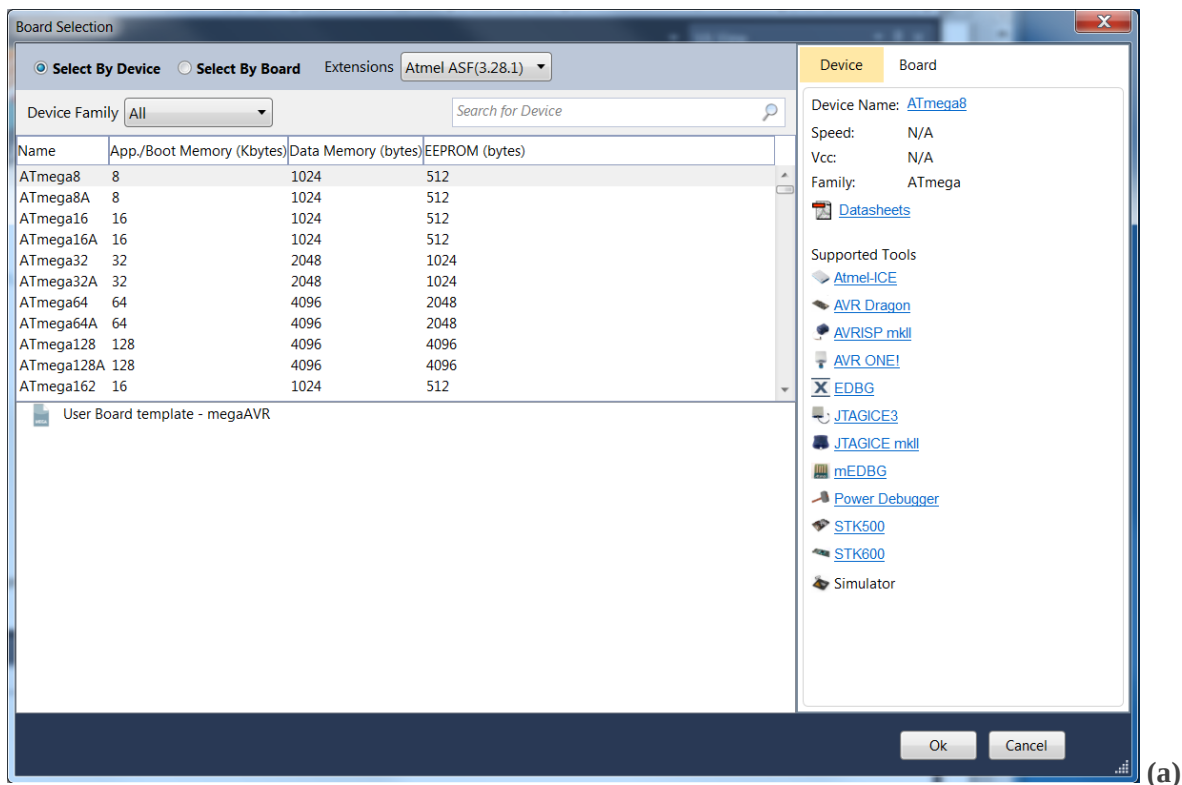
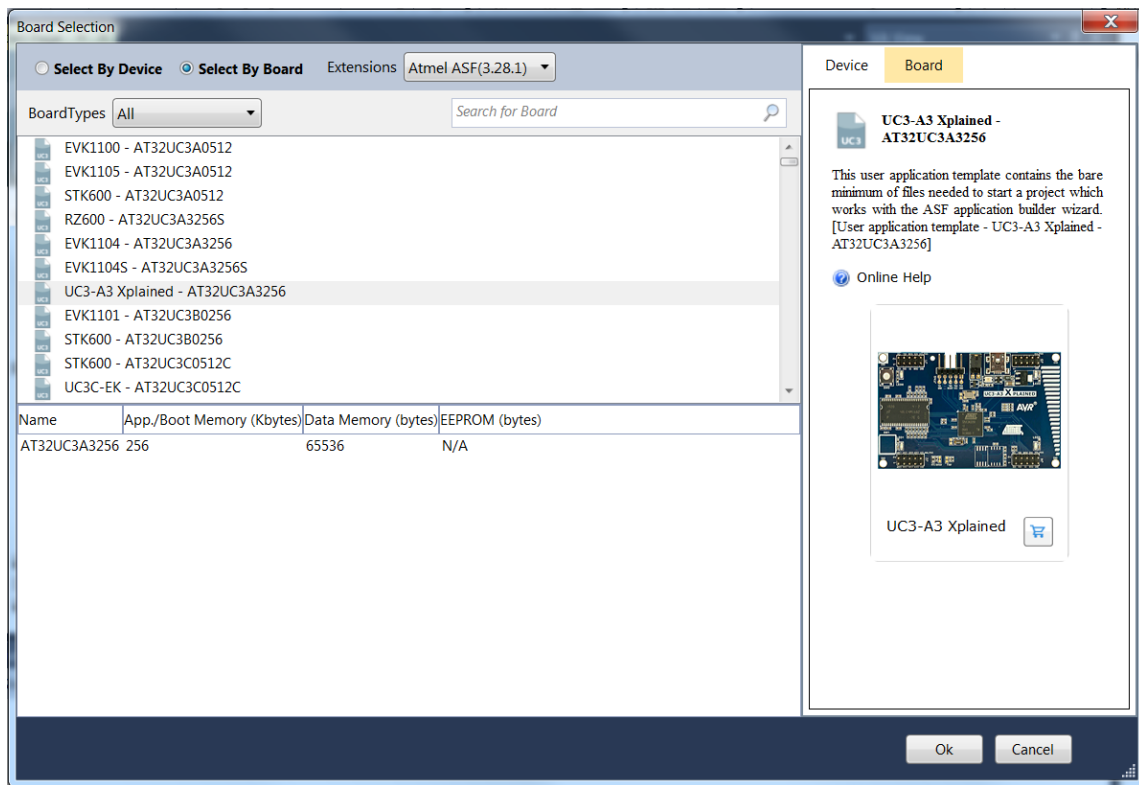


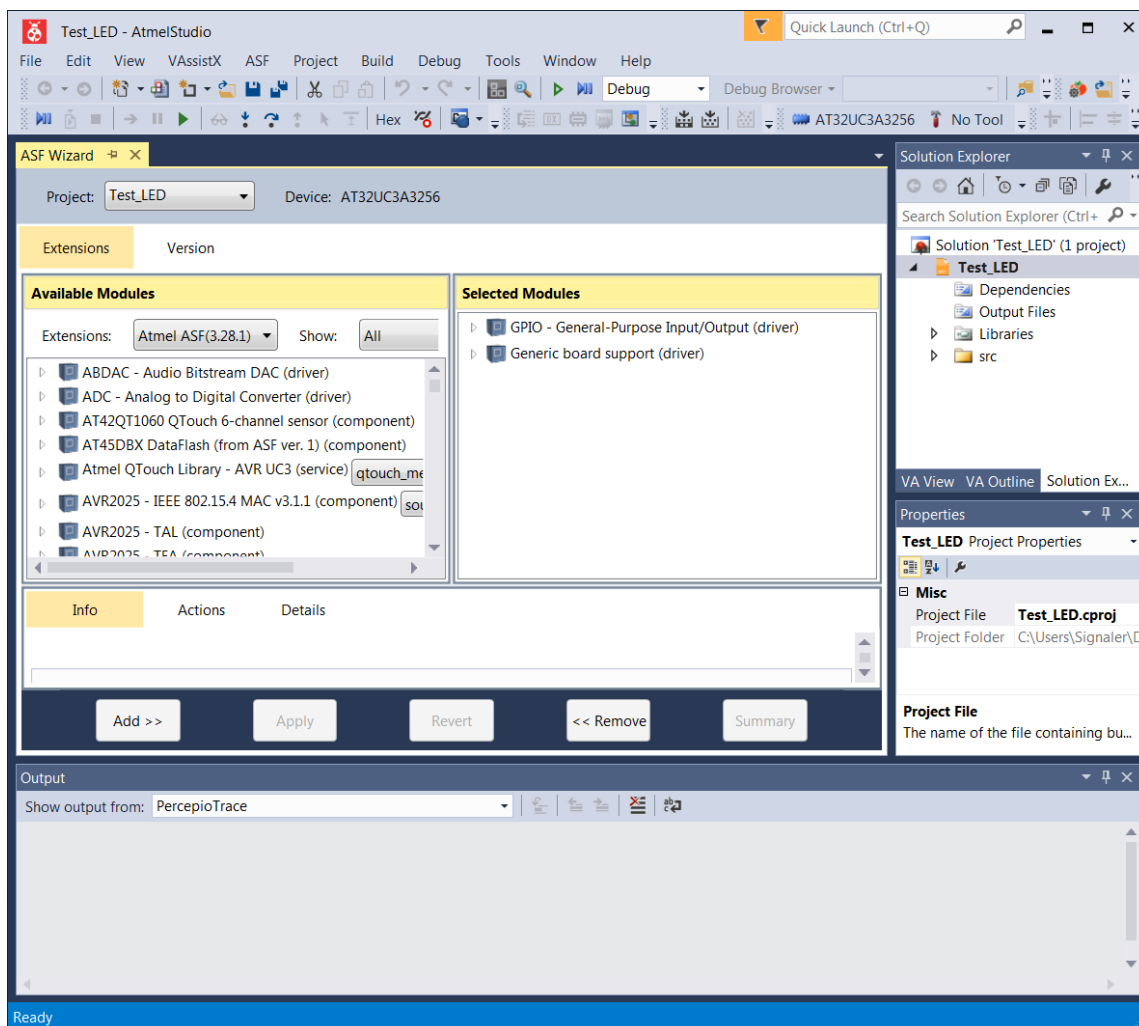
Fig. 6. New Project window



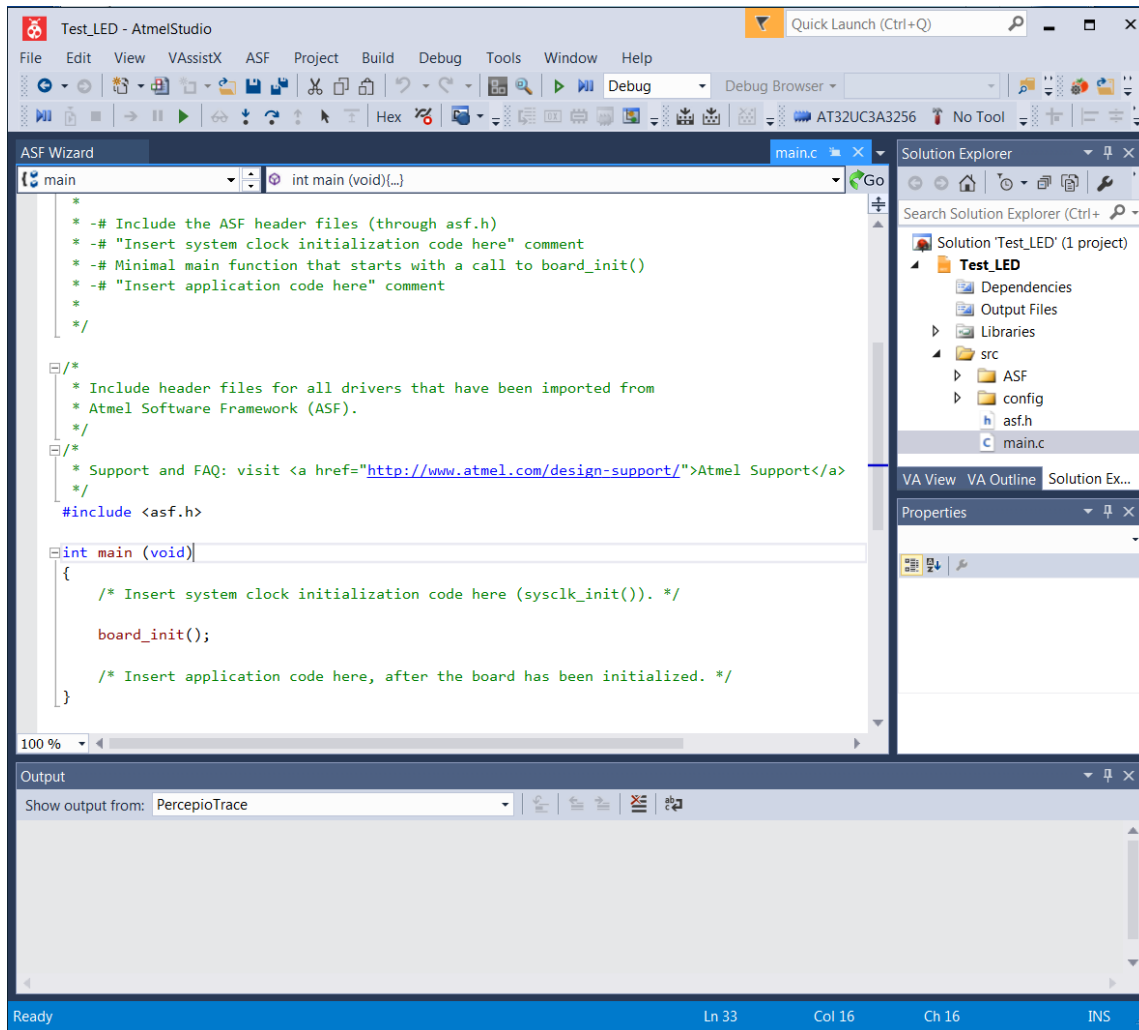


(b)

Fig. 7. Selection of a target device from Installed Templates in the Board Selection window



(a)



(b)

Fig. 8. The created Solution window

Step 4: Write a program, e.g., the following C codes for toggling LED0 – LED3, which are available from the ASF

```
gpio_toggle_pin(LED0_GPIO);
gpio_toggle_pin(LED1_GPIO);
gpio_toggle_pin(LED2_GPIO);
gpio_toggle_pin(LED3_GPIO);
```

Build the solution (after saving it) by clicking on **Build** then on **Build Solution** (or press F7). After a while when Build succeeds, the Output window (the lower pane) shows

Build succeeded.

===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====

Now a program (solution) for toggling four LED's on the UC3-A3 Xplained board is ready. You need to program (upload or burn) it to the MCU, AT32UC3A3256, on the evaluation board. When this is done, the program run on the kit should set on all the four LED's.

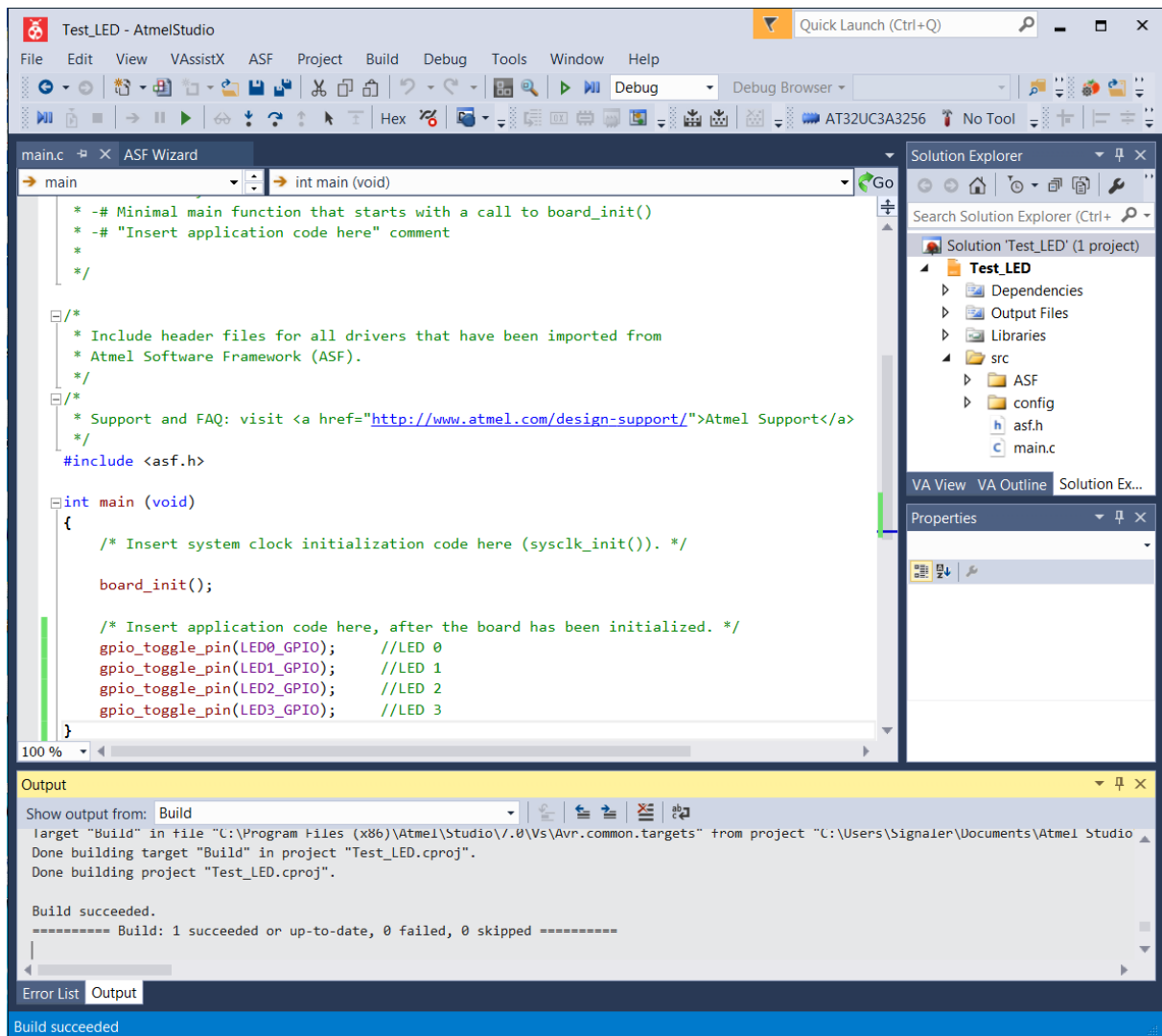


Fig. 9. Write and Build the example solution: Test_LED

4. batchisp

FLIP – Flexible In-system Programmer is Atmel’s software support in-system programming of flash devices through RS232, USB or CAN, and can run on Windows 9x, Windows ME, WindowsNT, Windows2000, WindowsXP, Windows Vista, Windows 7, Windows 8 and Linux x86. FLIP 3.4.7 is the latest version (requires Java Runtime Environment).

Batchisp (part of the FLIP) is a command line tool for in-system programming (ISP), and used for programming the UC3 chips via USB DFU Bootloader, in other words for loading a new program/firm-ware onto a MCU.

USB DFU (Device Firmware Upgrade) is an official USB device class specification of the USB Implementers Forum.

The batchisp command code is

```
batchisp -device at32uc3a3256 -hardware usb -operation erase f memory flash
blankcheck loadbuffer yourfilename.hex program verify start reset 0
```

which has the following means

- **erase f** – removes the flash, but not the bootloader section
- **memory flash** – selects flash as a target memory for the following commands
- **blankcheck** – checks if the flash is empty
- **loadbuffer yourfilename.hex** – loads yourfilename.hex file into the buffer
- **program** – programs the target memory with the data in the buffer

- **verify** – tells if there were any errors
- **start** – starts the board

The program “yourfilename.hex” (e.g., Project_1.hex in Fig. 10) is often located in the folder “-\\SolutionName\\ProjectName\\Debug\\” (e.g., “-\\Test_LED\\Project_1\\Debug\\” in Fig. 10). When the program is successfully loaded to the MCU, one can see the log information shown in Fig. 10.

Note that each time when you are loading a new program (firmware) to the MCU, you need to invoke Bootloader by pushing the physical push button (SW0) during powerup.

```

C:\WINDOWS\system32\CMD.exe
kcheck loadbuffer Project_1.hex program verify start reset 0
Running batchisp 1.2.5 on Sun Feb 10 16:05:45 2013

AT32UC3A3256 - USB - USB/DFU

Device selection..... PASS
Hardware selection..... PASS
Opening port..... PASS
Reading Bootloader version..... PASS    1.0.3
Erasing..... PASS
Selecting FLASH..... PASS
Blank checking..... PASS    0x00000 0x3ffff
Parsing HEX file..... PASS    Project_1.hex
WARNING: The user program and the bootloader overlap!
Programming memory..... PASS    0x00000 0x026ff
Verifying memory..... PASS    0x00000 0x026ff
Starting Application..... PASS    RESET    0

Summary: Total 11 Passed 11 Failed 0
C:\Documents and Settings\HA\My Documents\Atmel Studio\Test_LED\Project_1\Debug>

```

Fig. 10.

5. PuTTY and USART

To be able to communicate between the MCU and a PC (e.g., send signal's FFT or characters from the MCU to the PC), one can use serial communication via USART. A USART is a piece of computer hardware that translates data between parallel and serial forms. USARTs are commonly used in conjunction with communication standards such as RS-232, RS-422, RS-485 or EIA.

In AVR microcontrollers, USART is based on the RS-232 standard. For the communication between the MCU and a PC, a RS232-to-USB cable (simply called USB-RS232 cable, as shown in Fig.16) together with a corresponding driver is used to connect the MCU and the PC. Complete information on the USART drivers for AVR UC3-A3 microcontroller is available in the following link http://asf.atmel.com/docs/latest/uc3a3_a4/html/group_group_avr32_drivers_usart.html

PuTTY is an SSH and telnet client that can be used to receive and display on PC the digital data and other info sent from the MCU via USART. To make sure that this works, a correct configuration on PuTTY is necessary.

Configure PuTTY in “SSH → Serial” in the following manner in Fig. 11. Note that the serial to USB cable with a driver installed needs to be connected, and the serial port (on the PC) used by the cable is not always fixed each time when the cable is connected. Please check “Ports (COM & LPT)” in “Device Manager” when the cable is connected, and then give the correct COM port.

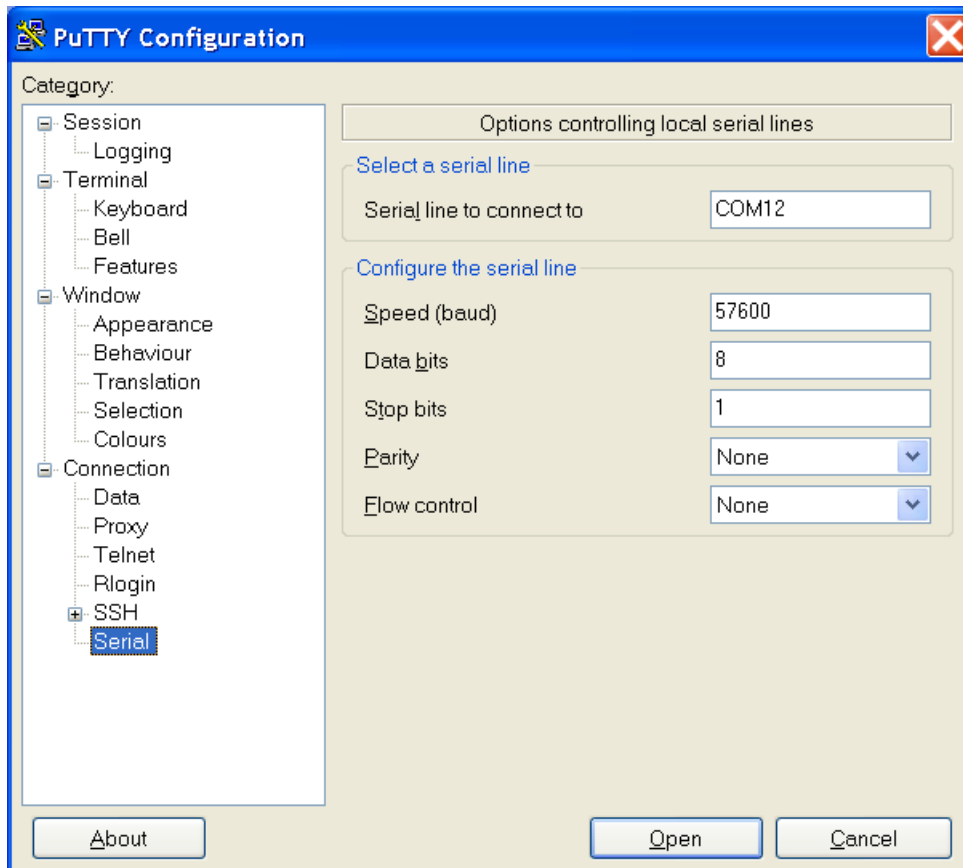


Fig. 11.

Give a name (e.g., User_UART in the current example) in “Session → Saved Sessions” in which “Serial” is chosen, and then Save the session

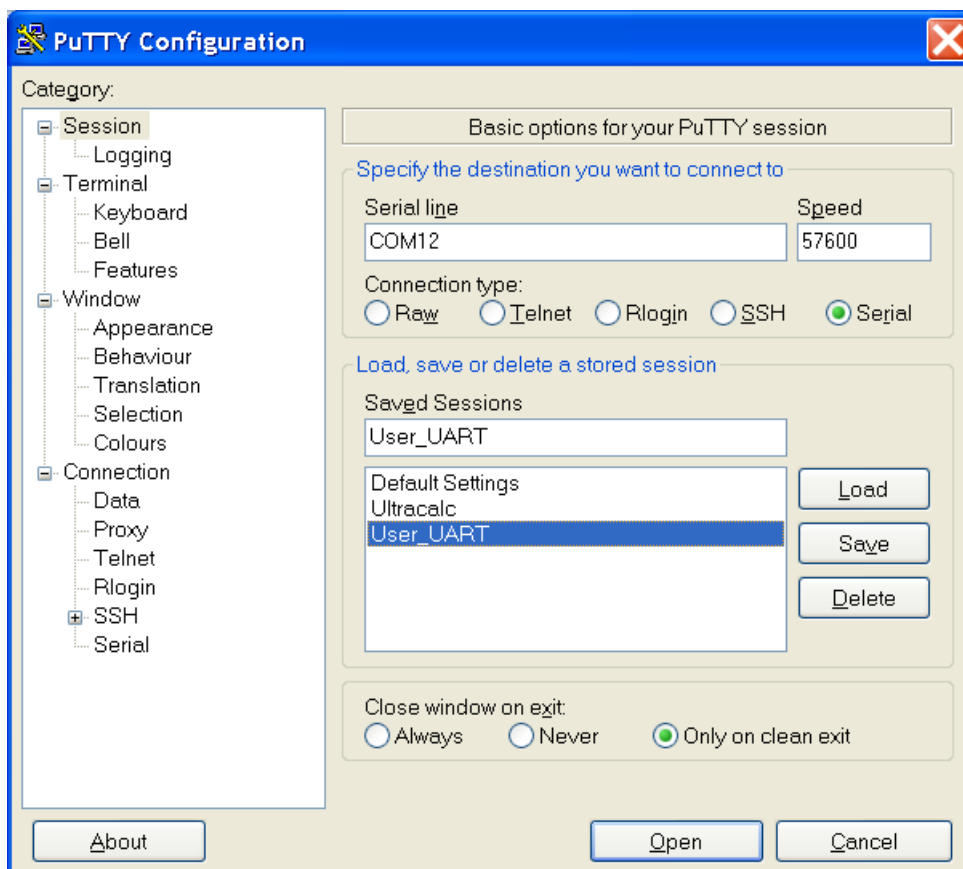


Fig. 12.

When this is done, press on Open. Then the following window appears (Fig. 13). The information sent from the evaluation kit via USART is displayed in the window, and the information from the PC can also be sent to the MCU through the window.

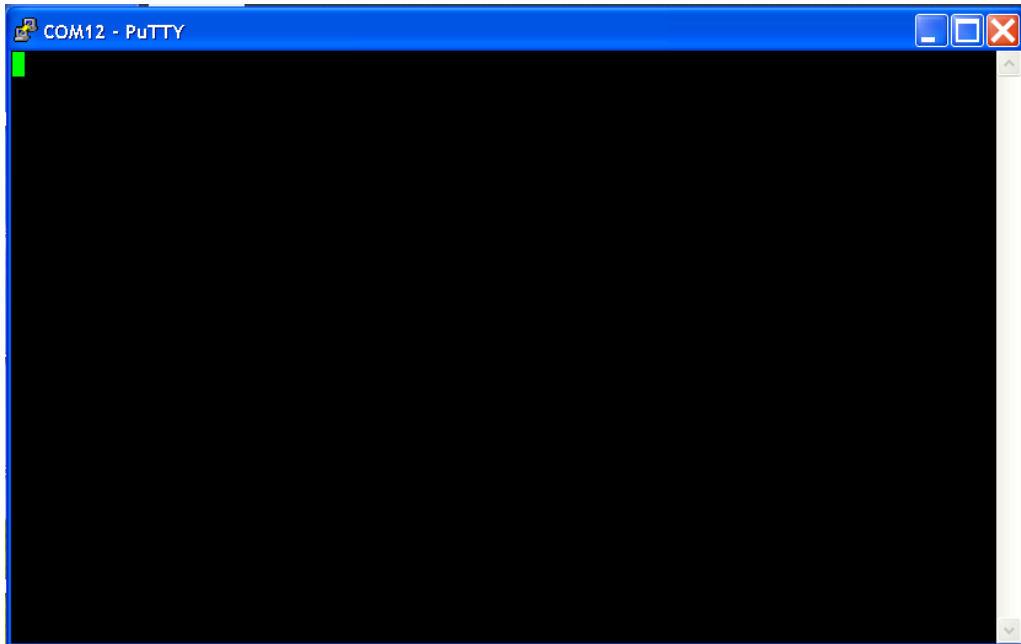


Fig. 13.

We are now creating a project (a program) to test USART with PuTTY, following the same steps to *create* and *build* the project Teste_LED.

The program is named as Test_USART, with the codes as follows

```
#include <asf.h>

// The options for initializing Usart:
// Baudrate 57600, Normal Channel Mode, 8 bit length
// No parity, 1 Stop bit, No flow control;
// Putty or MATLAB must have a Baudrate of 57600

static usart_options_t usart_opt = {
    .baudrate      = 57600,
    .channelmode   = USART_NORMAL_CHMODE,
    .charlength    = 8,
    .paritytype    = USART_NO_PARITY,
    .stopbits      = USART_1_STOPBIT,
};

int main (void)
{
    board_init();

    // Insert application code here, after the board has been initialized.
    sysclk_init();
    pm_switch_to_osc0(&AVR32_PM, FOSCO, OSC_STARTUP_0);    //pm - Power manager Clock to OSC0
    for Usart

        // USART init
        gpio_enable_module_pin(USART_TXD_PIN, USART_TXD_FUNCTION);    // Initial USART TX and RX
pins
        gpio_enable_module_pin(USART_RXD_PIN, USART_RXD_FUNCTION);

        usart_init_rs232(USART, &usart_opt, FOSCO);    // Initial USART for RS232.
        usart_write_line(USART, "U");    // Ascii for "U" = 0x55; Can read without invert.

    while (1)
```

```

{
    gpio_toggle_pin(LED0_GPIO);           // All LED pins initialed in board_init();
    gpio_toggle_pin(LED1_GPIO);
    gpio_toggle_pin(LED2_GPIO);
    gpio_toggle_pin(LED3_GPIO);
    usart_write_line(USART, "U");
}
}

```

To support USART, one needs to include the following services or drivers:

- System Clock Control (service)
- PM – Power Manager (driver)
- USART – Universal Synchronous/Asynchronous Receiver/Transmitter (driver)

To this end, in the **ASF Wizard** pane (as shown in Fig. 8(a)), find the above services or drivers in **Available Modules**, and then add (Add >>) and apply (Apply). After this, the **ASF Wizard** pane looks like Fig. 14.

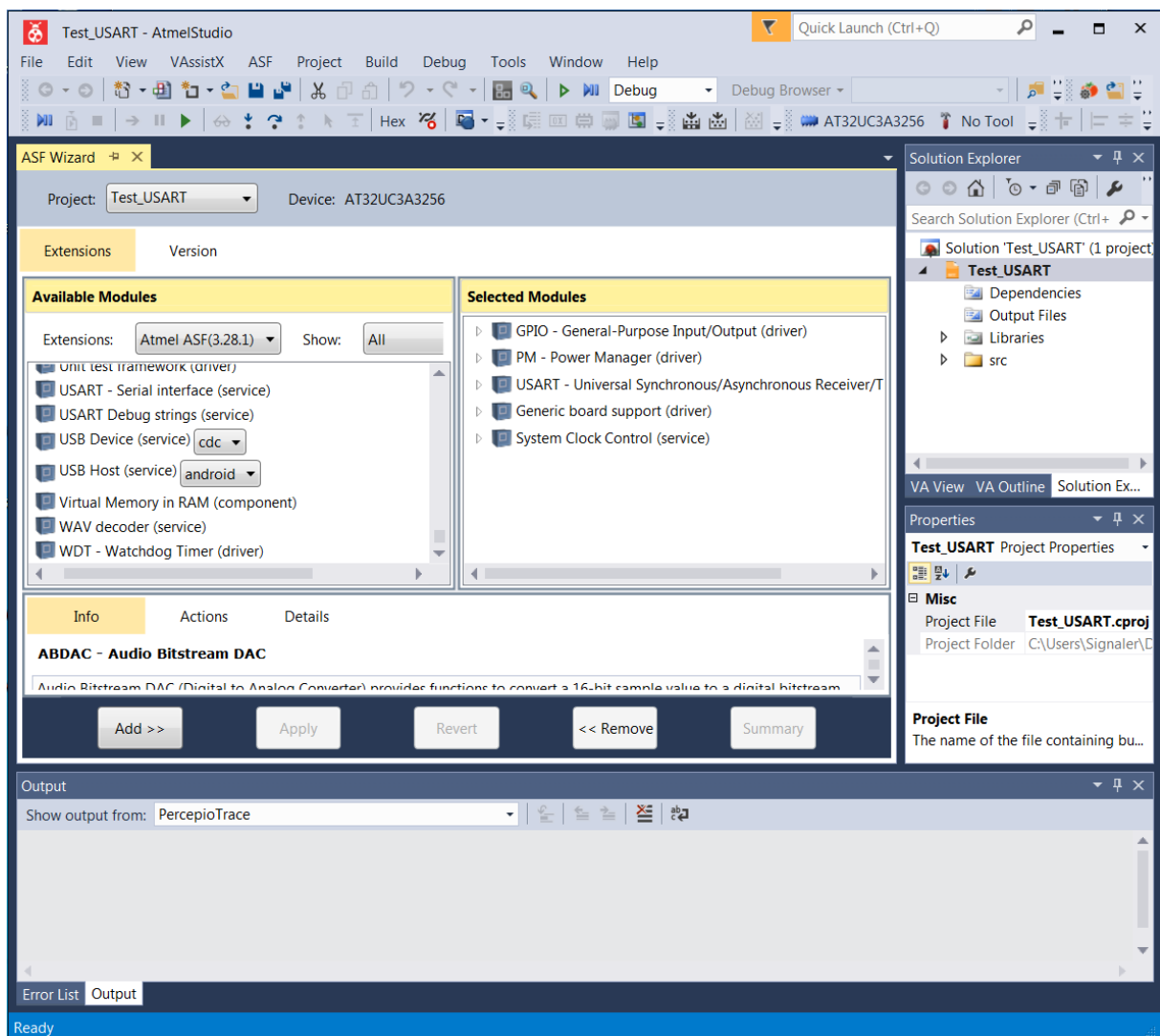


Fig. 14.

In the upper right pane in Fig. 14, click on src, and choose main.c. Then write the above-given C code in main.c, and build (F7) the project. The Test_USART window appears as shown in Fig. 15.

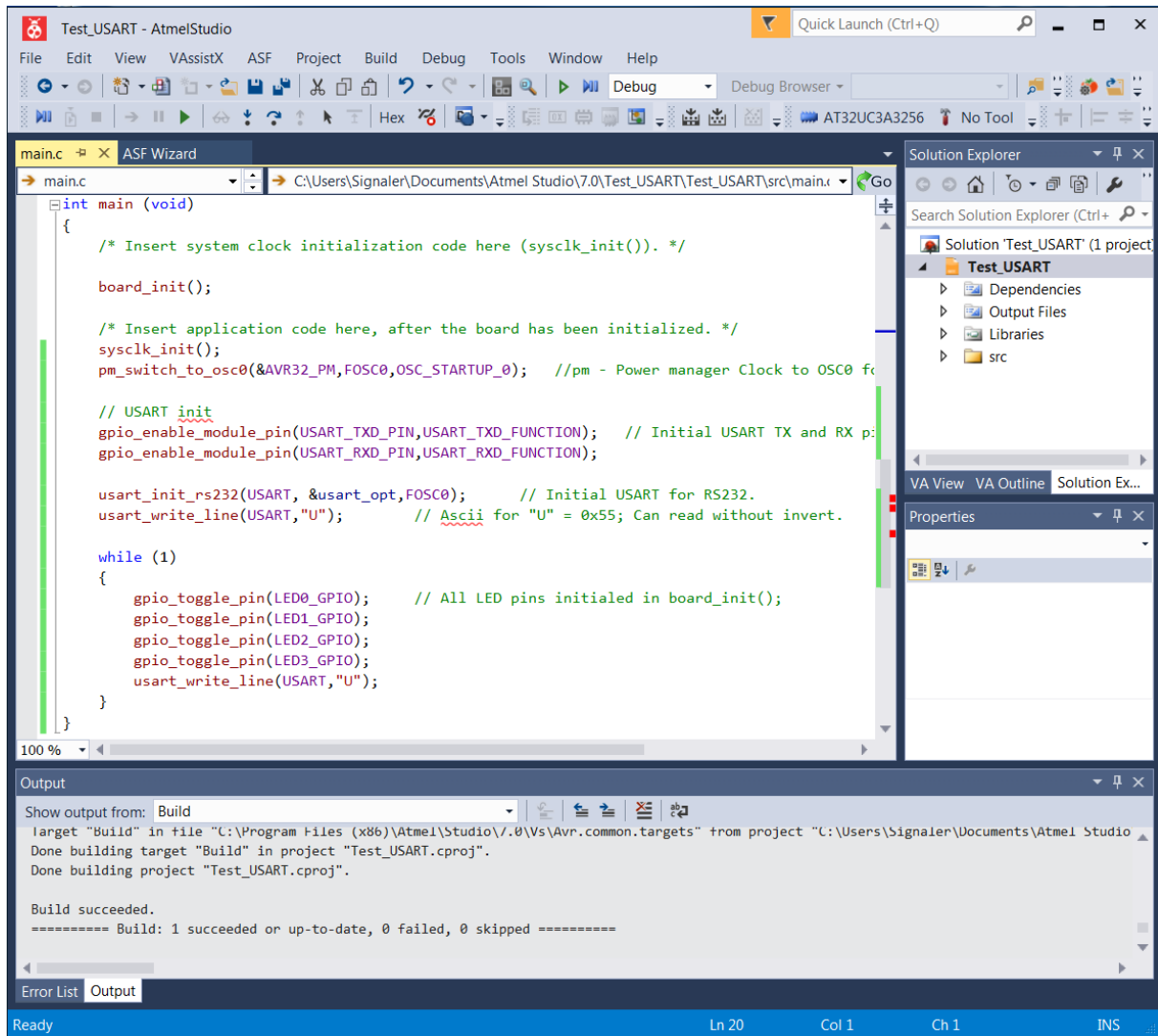


Fig. 15.

To see how Test_USART works, do the following

- Connect the USB cable to the UC3A3 Xplained evaluation kit (the USB connector)
- Connect the white and green wires of the USB-RS232 cable, respectively, to the TXD and RXD pins of header J4 on the kit (Fig. 16). The detail on RS232 is given in Appendix B.
Do not connect the VCC and GND lines (that are 5 V power) to any pins on the UC3A3 kit, to prevent damaging the kit!!!
- Run PuTTY

Then one can see letters “UUU...” shown on the Command Prompt window.

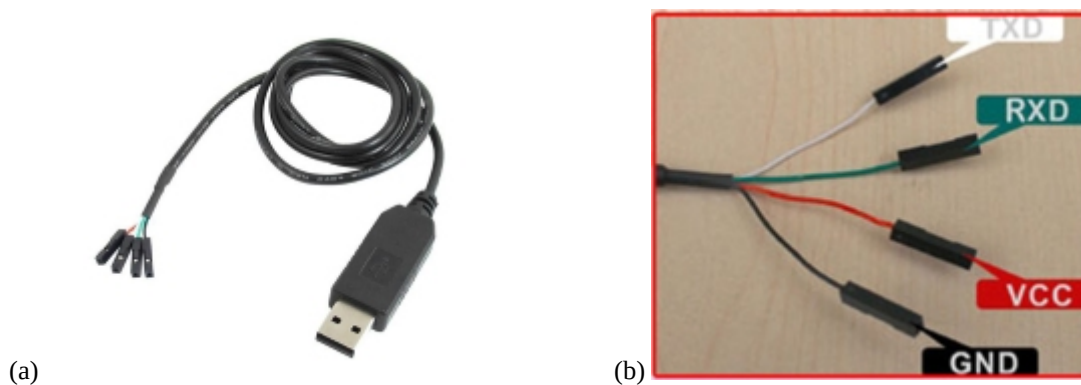


Fig. 16. (a) The USB-RS232 cable and (b) its RS232 pinout.

If one wants to make LEDs be blinking, he/she can use delaying functions, e.g., `delay_us( )`, `delay_ms( )`, `delay_s( )`. To this purpose, one needs to add the following service through **ASF Wizard**:

- Delay routines (service)

For LEDs to blink every one second, add the following line in the program `Test_USART` as in Fig. 17
`delay_ms(1000);`

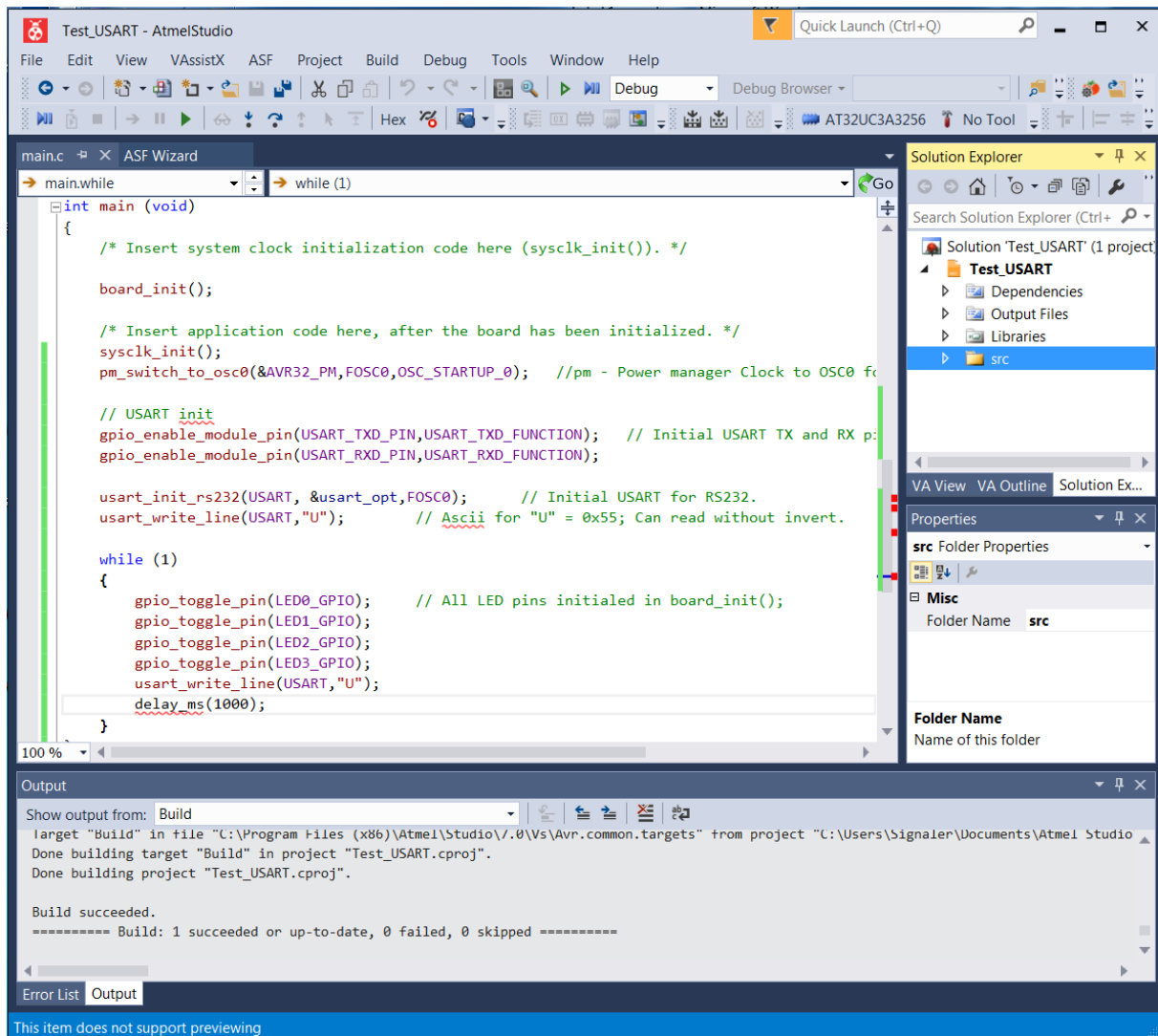


Fig. 17.

Let's have a look at an extended USART example in which more USART functions are used, e.g.,

- `usart_getchar(USART); // receive a letter from Putty on the PC`
- `usart_putchar(USART, int c); // send a letter c to Putty`

Complete information on the USART functions (drivers) for the kit is available in the following link
http://asf.atmel.com/docs/latest/uc3a3_a4/html/group_group_avr32_drivers_usart.html

The C codes for the USART example are given below

```
int main (void)
{
    board_init();

    // Insert application code here, after the board has been initialized.
    sysclk_init();
    pm_switch_to_osc0(&AVR32_PM, FOSC0, OSC_STARTUP_0); //pm - Power manager Clock to OSC0
    for Usart
```

```

// Assign GPIO to USART
gpio_enable_module_pin(USART_TXD_PIN, USART_TXD_FUNCTION);    // Initial USART TX and RX
pins
gpio_enable_module_pin(USART_RXD_PIN, USART_RXD_FUNCTION);

usart_init_rs232(USART, &usart_opt, FOSCO);    // Initialize USART in RS232 mode

usart_write_line(USART, "\r\n"); // \r is a carriage return and \n is a line feed
usart_write_line(USART, "***** Test USART *****");

char test_c;
char test_str[5];

while (1)
{
    gpio_toggle_pin(LED0_GPIO);    // All LED pins initialed in board_init();
    gpio_toggle_pin(LED1_GPIO);
    gpio_toggle_pin(LED2_GPIO);
    gpio_toggle_pin(LED3_GPIO);

    usart_write_line(USART, "\r\nHello, this is AVR UC3-A3 Xplained saying hello!
    (press enter)\r\n");
    while (usart_get_echo_line(USART) == USART_FAILURE); // Get and echo characters
    until end of line.

    usart_write_line(USART, "\r\nPlease type in a letter: ");
    test_c=usart_getchar (USART); // receive a letter from Putty Window
    usart_write_line(USART, "\r\nYou have typed in a letter: ");
    usart_putchar(USART, test_c); // display the letter received by usart_getchar()

    usart_write_line(USART, "\r\n\nPlease type in a word with 5 characters: ");
    for (int n = 0; n<5; n++) { //get a string via USART
        test_str[n]=usart_getchar(USART);
        usart_putchar(USART, test_str[n]);
    }
    usart_write_line(USART, "\r\nYou have typed in a word: ");
    for (int n = 0; n<5; n++) usart_putchar(USART, test_str[n]); // display the
    string received by usart_getchar() in the for loop

    while (usart_get_echo_line(USART) == USART_FAILURE); // Get and echo characters
    until end of line.
    usart_write_line(USART, "\r\n\nGoodbye. \n");
}
}

```

6. Exercises

After finishing the above tasks, use the C codes on the next page to do the following exercises

- (i). Use the function `User_Usart_int2str(uint32_t User_int32)` to display on Putty the results of $12340/10$, $1/10*12340$, $0.1*12340$, i.e., `User_Usart_int2str(1/10*12340)`, etc., and see if all the results are the same;
- (ii). Use the function `User_Usart_num2str(float User_num)` to display on Putty the numbers 12345.678 and 12345.6789, and see if they are the same
- (iii). (Optional) Try to write a function that can use Putty to input numbers, do simple math operations like +, -, * and /, and display the result on the Putty, e.g., calculate $12345.678 / 2$ and display the calculated result on the Putty,

```

static uint16_t User_Binbcd(uint16_t User_Bin);
static void User_Usart_int2str(uint32_t User_int32);
static void User_Usart_num2str(float User_num);

/*-----*/
/*  convert int to ascii,  e.g., 1 => '0x31'                                     */
/*-----*/
static uint16_t User_Binbcd(uint16_t User_Bin)
{
    User_Bin=User_Bin&0x0f;
    User_Bin=User_Bin+0x30;
    return User_Bin;
}

/*-----*/
/*  Convert a number to ascii array and put to the usart buffer end with ',' , */
/*  e.g., 123 => {0x30,0x30,0x31,0x32,0x33,',','}                               */
/*-----*/
static void User_Usart_int2str(uint32_t User_int32)
{
    usart_putchar(USART,User_Binbcd( User_int32/10000)) ;
    usart_putchar(USART,User_Binbcd( User_int32%10000/1000)) ;
    usart_putchar(USART,User_Binbcd( User_int32%1000/100)) ;
    usart_putchar(USART,User_Binbcd( User_int32%100/10)) ;
    usart_putchar(USART,User_Binbcd( User_int32%10)) ;
    //          usart_write_line(USART,"");
}

/*-----*/
/*  convert a number xxxxx.yyy to ascii,                                     */
/*-----*/
static void User_Usart_num2str(float User_num)
{
    uint32_t n;
    float y;

    User_Usart_int2str(User_num);
    usart_write_line(USART,".");
    y = (User_num - (int)User_num);

    for (n=1;n<4;n++)
    {
        y = y*10;
        usart_putchar(USART,User_Binbcd( (int)y%10)) ;
    }
}

```

Appendix A. Block diagram and key parameters of AT32UC3A3256 microcontroller

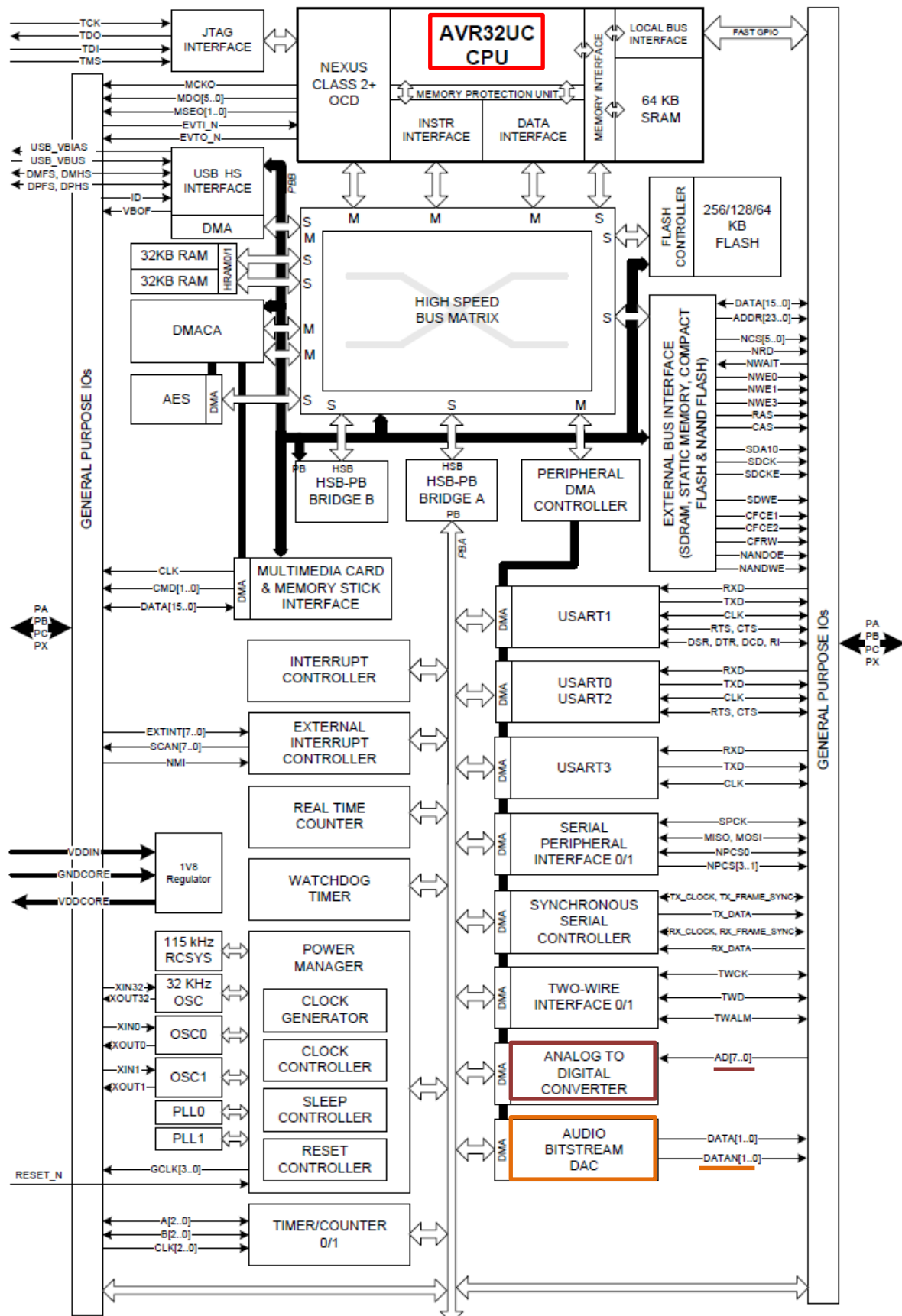


Fig. A-1. Block diagram of AVR UC3 microcontroller [M10].

Table 2-1. Configuration Summary

Feature	AT32UC3A3256/128/64	AT32UC3A4256/128/64
Flash	256/128/64 KB	
SRAM	64 KB	
HSB RAM	64 KB	
EBI	Full	Nand flash only
GPIO	110	70
External Interrupts	8	
TWI	2	
USART	4	
Peripheral DMA Channels	8	
Generic DMA Channels	4	
SPI	2	
MCI slots	2 MMC/SD slots	1 MMC/SD slot + 1 SD slot
High Speed USB	1	
AES (S option)	1	
SSC	1	
Audio Bitstream DAC	1	
Timer/Counter Channels	6	
Watchdog Timer	1	
Real-Time Clock Timer	1	
Power Manager	1	
Oscillators	PLL 80-240 MHz (PLL0/PLL1) Crystal Oscillators 0.4-20 MHz (OSC0/OSC1) Crystal Oscillator 32 KHz (OSC32K) RC Oscillator 115 kHz (RCSYS)	
10-bit ADC number of channels	1 8	
JTAG	1	
Max Frequency	66 MHz	
Package	LQFP144, TFBGA144	VFBGA100

Key parameters for AT32UC3A3256

<u>Parameter</u>	<u>Value</u>
<u>Flash (Kbytes):</u>	<u>256 Kbytes</u>
<u>Pin Count:</u>	<u>144</u>
<u>Max. Operating Frequency:</u>	<u>66 MHz</u>
<u>CPU:</u>	<u>32-bit AVR</u>
<u># of Touch Channels:</u>	<u>32</u>
<u>Max I/O Pins:</u>	<u>110</u>
<u>Ext Interrupts:</u>	<u>110</u>
<u>USB Transceiver:</u>	<u>1</u>

USB Speed:	Hi-Speed
SPI:	6
TWI (I2C):	2
UART:	4
ADC channels:	8
ADC Resolution (bits):	10
ADC Speed (ksps):	384
DAC Channels:	2
DAC Resolution (bits):	16
SRAM (Kbytes):	128
External Bus Interface:	1
DRAM Memory:	sdram
I/O Supply Class:	3.0 to 3.6
Operating Voltage (Vcc):	3.0 to 3.6
Timers:	6
Output Compare channels:	18
Input Capture Channels:	12
PWM Channels:	12
32kHz Real Time Counter (RTC):	Yes

References

- M6. Steven F. Barrett and Daniel J. Pack, *Atmel AVR Microcontroller Primer Programming and Interfacing*, 2nd Ed, Morgan & Claypool Publ., 2012.
<http://www.morganclaypool.com/doi/pdf/10.2200/S00427ED1V01Y201206DCS039>
- M7. * Steven F. Barrett and Daniel J. Pack, *Microcontrollers Fundamentals for Engineers and Scientists*, Morgan & Claypool Publ., 2006. (*For those who are new to Embedded Systems and have not taken any courses on Computer Systems or Embedded Systems).
<http://www.morganclaypool.com/doi/pdf/10.2200/S00025ED1V01Y200605DCS001>
- M8. Atmel, *Atmel AVR32930: UC3-A3 Xplained Getting Started Guide*,
<http://www.atmel.com/Images/doc32163.pdf>
- M9. Atmel, *AVR32918: UC3-A3 Xplained Hardware User's Guide*,
<http://www.atmel.com/Images/doc32159.pdf>
- M10. Atmel 32-bit AVR® Microcontroller <http://www.atmel.com/Images/doc32072.pdf>
- M11. Atmel, *Atmel AVR4029: Atmel Software Framework - Getting Started*,
<http://www.atmel.com/Images/doc8431.pdf>
- M12. Atmel, *Atmel AVR4030: Atmel Software Framework - Reference Manual*,
<http://www.atmel.com/Images/doc8432.pdf>
- M13. Atmel, *AVR32718: AT32UC3 Series Software Framework DSPLib*
<http://www.atmel.com/Images/doc32076.pdf>
- M14. Atmel, *AVR32765: AVR32 DSPLib Reference Manual*
<http://www.atmel.com/Images/doc32120.pdf>
- M15. <http://www.atmel.com/>